

Equazione del Calore 2D e Riordinamento nelle Fattorizzazioni Cholesky

Documento di Progetto
Informatica con Laboratorio – A.A. 2025/2026

Giosuè Aiello

1 Impostazione del problema

L'obiettivo del progetto è la risoluzione numerica dell'equazione del calore in regime stazionario in due dimensioni spaziali. Il dominio di calcolo è il quadrato unitario $\Omega = [0, 1]^2$, e si cerca la funzione di temperatura $u : \Omega \rightarrow \mathbb{R}$ che soddisfa l'equazione ellittica:

$$\kappa \left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2} \right) + f(x, y) = 0, \quad (x, y) \in \Omega, \quad (1)$$

con condizioni al contorno di Dirichlet sul bordo $\partial\Omega$:

$$u(x, y) = u_0(x, y), \quad (x, y) \in \partial\Omega.$$

I parametri fisici del problema sono: coefficiente di diffusività termica $\kappa = 0.01$ e termine sorgente $f(x, y) = e^{-10(x^2+y^2)}$, che modella una sorgente di calore concentrata in prossimità dell'origine con decadimento gaussiano. Nel caso base si considera $u_0 \equiv 0$ (condizioni omogenee).

Discretizzazione alle differenze finite

Si introduce una griglia cartesiana uniforme di $(N + 2)^2$ punti (x_i, y_j) con $i, j = 0, \dots, N + 1$ e passo $h = 1/(N + 1)$, dove $x_i = i \cdot h$ e $y_j = j \cdot h$. I punti di bordo hanno temperatura fissata e non contribuiscono come incognite. Le N^2 incognite del problema sono i valori $u_{i,j} \approx u(x_i, y_j)$ nei punti *interni* ($1 \leq i, j \leq N$).

Approssimando le derivate seconde con lo stencil a cinque punti, l'equazione (1) valutata nel punto interno (x_i, y_j) diventa:

$$\frac{\kappa}{h^2} (u_{i-1,j} + u_{i+1,j} + u_{i,j-1} + u_{i,j+1} - 4u_{i,j}) + f(x_i, y_j) = 0. \quad (2)$$

L'insieme delle equazioni (2) costituisce il sistema lineare $Au = b$, dove $A \in \mathbb{R}^{N^2 \times N^2}$ è sparsa (al più 5 elementi non nulli per riga), simmetrica e definita negativa.

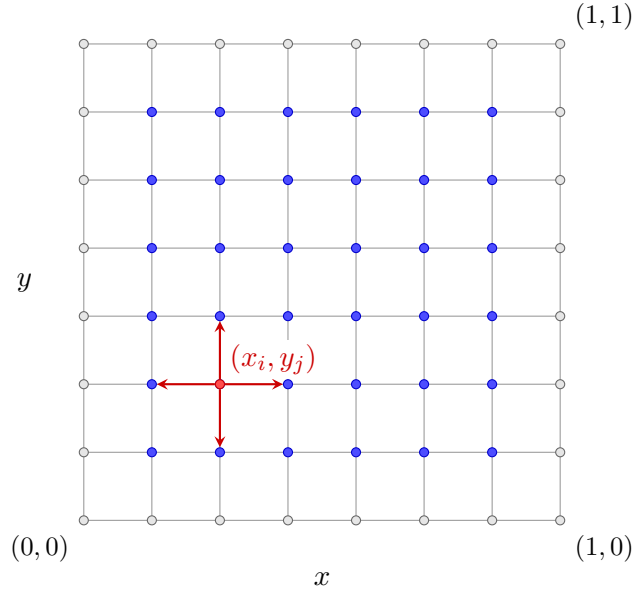


Figura 1: Griglia di discretizzazione: nodi di bordo (grigio), nodi interni (blu) e i quattro vicini dello stencil per il nodo (x_i, y_j) (freccie rosse).

Fattorizzazione di Cholesky e fill-in

Poiché $-A$ è simmetrica definita positiva (SPD), si calcola la fattorizzazione di Cholesky $-A = LL^T$ con L triangolare inferiore sparsa. Durante la fattorizzazione si generano elementi non nulli in posizioni che erano zero in $-A$: questo fenomeno è detto *fill-in* e dipende fortemente dall'ordine con cui le incognite sono enumerate.

Con l'ordinamento naturale per righe la larghezza di banda di A è $\Theta(N)$ e il fill-in di L è $O(N^3)$. La tecnica di *nested dissection* riduce il fill-in a $O(N^2 \log N)$, con un risparmio sostanziale per valori grandi di N .

Nested dissection: approccio geometrico

La struttura di A è codificata dal *grafo di adiacenza* $G = (V, E)$: ogni nodo $v \in V$ corrisponde a una variabile $u_{i,j}$ e ogni arco $(u, v) \in E$ indica che $A_{uv} \neq 0$. Per la griglia regolare:

$$(x_i, y_j) \leftrightarrow (x_k, y_l) \iff (i = k, |j - l| = 1) \text{ oppure } (j = l, |i - k| = 1).$$

La *nested dissection* individua ricorsivamente un separatore V_S che divide V in due insiemi bilanciati V_1 e V_2 senza archi diretti tra loro. L'ordinamento risultante elenca prima V_1 , poi V_2 , infine V_S , in modo che il separatore appaia in fondo alla matrice, limitando la propagazione del fill-in.

In questo progetto si adotta un **approccio geometrico puro**: il separatore non è determinato esplorando il grafo di adiacenza, bensì è definito direttamente tramite le coordinate spaziali dei nodi. Scelto l'asse di taglio $c \in \{x, y\}$ e calcolato l'indice mediano $\hat{c}$, si pone:

$$V_1 = \{u_{i,j} \mid c < \hat{c}\}, \quad V_2 = \{u_{i,j} \mid c > \hat{c}\}, \quad V_S = \{u_{i,j} \mid c = \hat{c}\}.$$

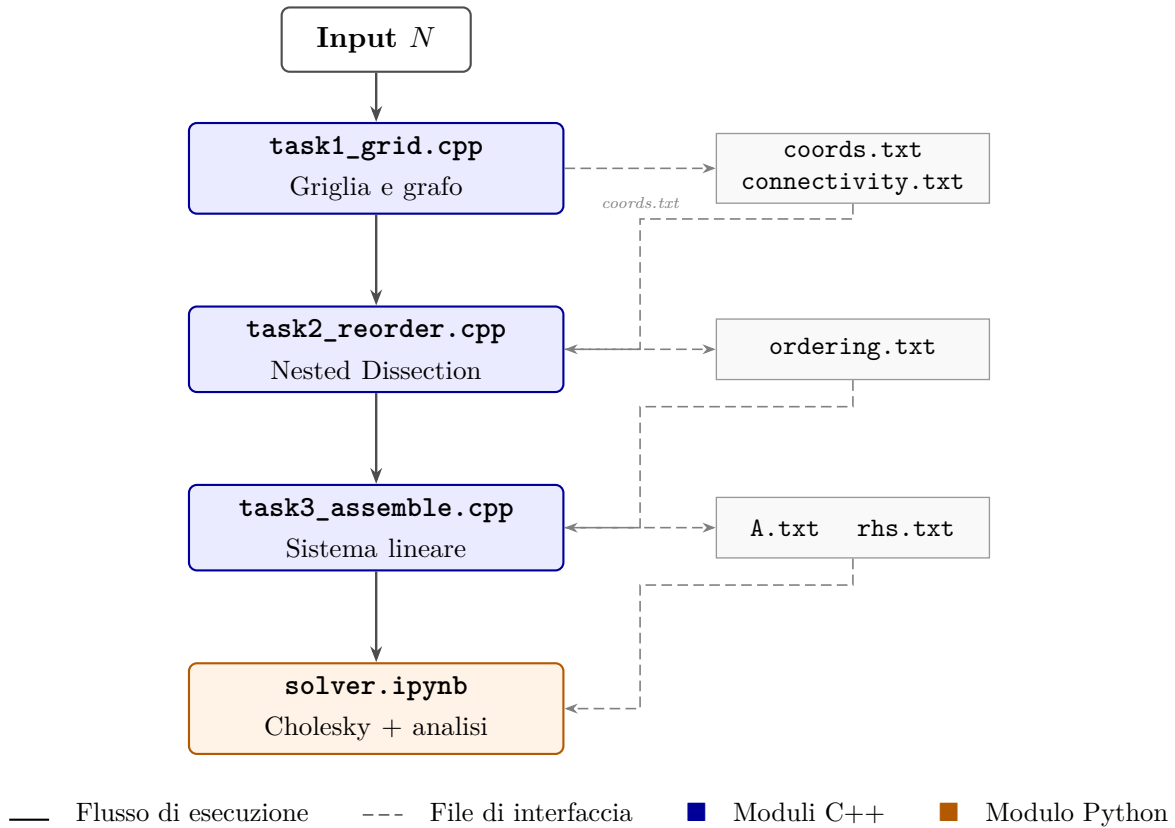
Il procedimento viene applicato ricorsivamente a V_1 e V_2 , alternando il taglio lungo x e y . Per griglie cartesiane regolari, il piano mediano è *al contempo* un separatore geometrico e un separatore topologico del grafo: non esistono archi di adiacenza tra V_1 e V_2 (i punti sono separati

da almeno un passo reticolare lungo l'asse di taglio), quindi l'approccio geometrico produce esattamente il medesimo separatore ottimale che si otterrebbe esplorando la topologia del grafo, senza il costo di tale esplorazione.

2 Architettura del progetto

Modulo principale: HeatEquationSolver

Il sistema viene concepito come un unico risolutore logico denominato `HeatEquationSolver`, articolato in quattro sotto-moduli specializzati. I tre moduli in C++ comunicano tra loro attraverso file di testo con formato fisso che fungono da interfaccia verificabile indipendentemente; il modulo Python costituisce il punto di ingresso per la fase numerica finale.



Le strutture dati fondamentali dell'intero sistema sono:

- **Liste di adiacenza** (`std::vector<std::vector<int>>`) nel Modulo 1 per la costruzione e scrittura del grafo di adiacenza.
- **Vettore di nodi geometrici** (`std::vector<Node>`) nel Modulo 2: `struct Node` contiene `id`, `i`, `j`, `x`, `y`; consente la nested dissection geometrica senza caricare in memoria la lista di adiacenza.
- **Stack esplicito** (`std::stack<StackItem>`) nel Modulo 2: sostituisce la ricorsione, evitando stack overflow su griglie $N \gg 1000$.
- **Vettori di permutazione** (`std::vector<int> perm`, `inv_perm`) per la mappatura bidirezionale tra ordinamento naturale e riordinato.
- **Formato sparso CSC** (`scipy.sparse.csc_matrix`) per la matrice A nel modulo Python, richiesto nativamente da CHOLMOD.

3 Specifiche dei moduli

I quattro sotto-moduli vengono descritti con livello di dettaglio crescente. Per ciascuno si indicano input, output, funzionalità richieste, strutture dati, invarianti, dipendenze e complessità attesa.

Modulo 1 – `task1_grid.cpp`: generazione della griglia e del grafo

Il Modulo 1 è responsabile della creazione della struttura geometrica del dominio e del relativo grafo di adiacenza. Costituisce il punto di ingresso dell'intera pipeline e non dipende da nessun altro modulo.

Formalizzazione matematica

Il dominio $\Omega = [0, 1]^2$ viene discretizzato con passo $h = \frac{1}{N+1}$. L'insieme dei nodi interni (le incognite) è:

$$V = \{(x_i, y_j) \mid x_i = i \cdot h, y_j = j \cdot h, \quad i, j \in \{1, \dots, N\}\}, \quad |V| = N^2.$$

Il grafo di adiacenza $G = (V, E)$ ha archi:

$$E = \left\{ \{u, v\} \subset V \mid \sqrt{(x_i - x_k)^2 + (y_j - y_l)^2} = h \right\}.$$

Ogni nodo interno ha al massimo 4 vicini (sinistra, destra, basso, alto). A ogni nodo viene assegnato un identificatore intero univoco tramite la mappa $\phi(i, j) = (i - 1) \cdot N + (j - 1)$.

Specifiche

- **Input:** Intero $N > 0$ passato da riga di comando; se assente o non valido, viene richiesto interattivamente.
- **Output:**
 - `output/coords.txt` – una riga per nodo: `id i j x y`
 - `output/connectivity.txt` – una riga per arco: `e u v` (dove `e` è l'indice progressivo dell'arco e $u < v$)
- **Funzionalità richieste:**
 - Parsing e validazione di N con fallback interattivo.
 - Generazione dei N^2 nodi con ID progressivo e coordinate spaziali (precisione `double`).
 - Costruzione della lista di adiacenza bidirezionale e scrittura degli archi senza duplicati ($u < v$).
 - Creazione della directory `output/` se non esiste.
- **Strutture dati:** `std::vector<std::vector<int>>` di dimensione N^2 , pre-allocata con `reserve(4)` per ciascun nodo.
- **Invarianti:**
 - La lista di adiacenza è sempre simmetrica: se $v \in \text{adj}[u]$ allora $u \in \text{adj}[v]$.
 - Ogni arco appare esattamente una volta in `connectivity.txt` (condizione $u < v$).
 - Il numero di righe in `coords.txt` è esattamente N^2 .

- **Dipendenze:** Nessuna. Questo modulo è il primo della pipeline.
- **Complessità attesa:** $\mathcal{O}(N^2)$ in tempo e spazio.

Logica implementativa (pseudo-codice)

```

1. Leggi N da argv; se non valido, chiedi da stdin
2. Crea cartella output/ se non esiste
3. Per i = 1 .. N:
4.   Per j = 1 .. N:
5.     id = (i-1)*N + (j-1)
6.     x = i/(N+1), y = j/(N+1)
7.     Scrivi (id, i, j, x, y) su coords.txt
8.     Se j < N: aggiungi arco bidirezionale (id, id+1)
9.     Se i < N: aggiungi arco bidirezionale (id, id+N)
10. Per ogni nodo u, per ogni vicino v:
11.   Se u < v: scrivi (e++, u, v) su connectivity.txt

```

Modulo 2 – `task2_reorder.cpp`: nested dissection geometrica

Il Modulo 2 costituisce il cuore algoritmico del progetto. Calcola una permutazione ottimizzata dei nodi tramite la *nested dissection geometrica*, al fine di minimizzare il fill-in durante la fattorizzazione di Cholesky.

Approccio geometrico vs. approccio su grafo

Esistono due famiglie di algoritmi per la nested dissection:

1. **Approccio su grafo** (es. bisection spettrale, METIS): esplora esplicitamente il grafo di adiacenza $G = (V, E)$, calcola autovettori del Laplaciano o partiziona tramite BFS/Kernighan-Lin. Richiede $\mathcal{O}(|E|) = \mathcal{O}(N^2)$ solo per costruire e visitare il grafo, più il costo delle euristiche di partizionamento (generalmente $\mathcal{O}(N^2\alpha(N))$ con α lenta ma non banale). Tali metodi sono necessari per griglie non strutturate o domini irregolari.
2. **Approccio geometrico** (adottato in questo progetto): ignora completamente la lista di adiacenza. Il separatore viene individuato tramite il *piano mediano* sull'asse corrente: dati gli indici interi (i, j) dei nodi, si calcola $\hat{c} = \lfloor (\min_c + \max_c)/2 \rfloor$ e si partiziona $V_S = \{v \mid c_v = \hat{c}\}$, $V_1 = \{v \mid c_v < \hat{c}\}$, $V_2 = \{v \mid c_v > \hat{c}\}$.

Perché l'approccio geometrico è superiore per griglie regolari. Per una griglia cartesiana uniforme $N \times N$ l'approccio geometrico è *equivalente* a quello su grafo in termini di qualità del separatore, ma *superiore* sotto ogni aspetto implementativo:

- **Correttezza del separatore:** il piano mediano $c = \hat{c}$ divide la griglia in due metà con al massimo $\lceil N/2 \rceil$ nodi ciascuna. Non esistono archi di adiacenza tra V_1 e V_2 (i nodi adiacenti si trovano a distanza reticolare h lungo l'asse di taglio, quindi il nodo con $c = \hat{c}$ funge da buffer). Il separatore geometrico *coincide* esattamente con il separatore topologico del grafo.
- **Complessità:** $\mathcal{O}(N^2 \log N)$ deterministico, senza euristiche. L'approccio su grafo richiede almeno $\mathcal{O}(N^2)$ per la sola lettura degli archi, più il costo dell'algoritmo di partizionamento.
- **Efficienza in cache:** accesso sequenziale al solo vettore di nodi (`coords.txt` $\rightarrow$ `std::vector<Node>`), nessuna lista di puntatori per adiacenze.

- **Semplicità e verificabilità:** l'invariante “ V_S è il piano mediano” è dimostrabile analiticamente; l'invariante di un separatore trovato per BFS su grafo non strutturato richiede verifica sperimentale.
- **Robustezza:** utilizzo di confronti su indici interi $((i, j))$ anziché coordinate floating-point, eliminando i rischi di arrotondamento.

Formalizzazione matematica

L'algoritmo riceve in input il vettore S di nodi con coordinate intere (i, j) e reali (x, y) , e individua ricorsivamente:

$$V_1 = \{u \in S \mid c_u < \hat{c}\}$$

$$V_2 = \{u \in S \mid c_u > \hat{c}\}$$

$$V_S = \{u \in S \mid c_u = \hat{c}\}$$

dove $c \in \{i, j\}$ è l'indice dell'asse di taglio e $\hat{c} = \lfloor (\min_{u \in S} c_u + \max_{u \in S} c_u) / 2 \rfloor$. L'ordinamento risultante è $V_1 \rightarrow V_2 \rightarrow V_S$, ricorsivo con alternanza degli assi.

Specifiche

- **Input:** `output/coords.txt` (prodotto dal Modulo 1). Il file `connectivity.txt` *non viene letto*: per l'approccio geometrico la struttura topologica del grafo è ridondante.
- **Output:** `output/ordering.txt` – una riga per posizione: `new_id old_id`
- **Funzionalità richieste:**
 - Lettura delle sole coordinate da `coords.txt`.
 - Partizionamento iterativo tramite `std::stack<StackItem>` in heap (evita stack overflow per $N \gg 1000$).
 - Calcolo del piano mediano con indici interi, con alternanza degli assi i/j ad ogni livello.
 - Verifica dell'invariante: `ordering.size() == N2`.
 - Salvataggio dell'ordinamento su `ordering.txt`.
- **Strutture dati:** `struct Node {id, i, j, x, y}; struct StackItem {vector<Node> S, int axis, bool is_separator}; std::stack<StackItem>; std::vector<int> ordering.`
- **Invarianti:**
 - Il vettore `ordering` è una permutazione valida di $\{0, \dots, N^2 - 1\}$.
 - I nodi di V_S compaiono sempre *dopo* i nodi di V_1 e V_2 alla stessa profondità.
 - L'asse di taglio si alterna ad ogni livello ($0 = i, 1 = j$).
- **Dipendenze:** Modulo 1 (solo `coords.txt`).
- **Complessità attesa:** $\mathcal{O}(N^2 \log N)$ in tempo; $\mathcal{O}(N^2)$ in spazio.

Logica implementativa (pseudo-codice)

```

1. Carica nodi da output/coords.txt -> vector<Node>
2. Inizializza stack con (tutti i nodi, asse=0)
3. Finche' lo stack non e' vuoto:
4.   Estrai item = (S, asse, is_separator)
5.   Se is_separator: aggiungi tutti i nodi di S a ordering; continua
6.   Se |S| == 0: continua
7.   Se |S| == 1: aggiungi S[0].id a ordering; continua
8.   Calcola min_val, max_val su asse corrente (indice i o j)
9.   mid_val = (min_val + max_val) / 2
10.  Partiziona S in V1 (val<mid), V2 (val>mid), VS (val==mid)
11.  next_axis = 1 - asse
12.  Push (VS, asse, true)  -- separatore: elaborato per ultimo
13.  Push (V2, next_axis, false)
14.  Push (V1, next_axis, false) -- elaborata per prima (cima stack)
15.  Verifica: ordering.size() == nodes.size()
16.  Scrivi output/ordering.txt: per k=0..N^2-1, scrivi "k ordering[k]"

```

Modulo 3 – task3_assemble.cpp: assemblaggio del sistema lineare

Il Modulo 3 costruisce il sistema lineare $Au = b$ che approssima l'equazione differenziale. Supporta sia l'ordinamento naturale che quello ottimizzato via il flag `-reorder`, e implementa la parte opzionale delle condizioni al bordo non omogenee.

Formalizzazione matematica

Applicando lo stencil a 5 punti con passo $h = 1/(N + 1)$ e diffusività κ , i coefficienti della matrice per ogni nodo k (nel nuovo ordinamento) sono:

$$\begin{aligned}
 A_{k,k} &= -\frac{4\kappa}{h^2} \\
 A_{k,k'} &= +\frac{\kappa}{h^2} \quad \text{per ogni vicino interno } k'
 \end{aligned}$$

Il termine noto è $b_k = -f(x_{i(k)}, y_{j(k)})$. Se il vicino k' cade sul bordo (condizione di Dirichlet), il suo valore u_0 è noto e viene incorporato nel termine noto:

$$b_k \leftarrow \frac{\kappa}{h^2} \cdot u_0(x_{\text{bordo}}, y_{\text{bordo}}).$$

Specifiche

- **Input:** Intero N e flag opzionale `-reorder` da riga di comando; `output/coords.txt`; `output/ordering.txt` (solo se `-reorder`).
- **Output:** `output/A.txt` (triplette $i \ j \ val$) e `output/rhs.txt` (una riga per valore di b).
- **Funzionalità richieste:**
 - Lettura di `coords.txt` e costruzione della mappa $\phi^{-1} : (i, j) \rightarrow id$.
 - Se `-reorder`: lettura di `ordering.txt` e costruzione di `perm` e `inv_perm`; altrimenti vettori identità.

- Menu interattivo per la scelta della condizione al bordo u_0 (selezione avviene una sola volta, prima del ciclo di assemblaggio).
- Assemblaggio dello stencil con gestione delle condizioni di Dirichlet.
- Scrittura di `A.txt` e `rhs.txt` con precisione floating-point a 10 cifre decimali.
- **Strutture dati:** `std::vector<int> perm, inv_perm` di dimensione N^2 ; `struct Triplet {int row, col; double val;}`; `std::vector<Triplet>` pre-allocato con `reserve(5*N*N)`; `std::vector<double> rhs`.
- **Invarianti:**
 - `perm` e `inv_perm` sono permutazioni inverse l'una dell'altra: `inv_perm[perm[k]] == k` per ogni k .
 - La matrice A è simmetrica: se la tripletta (k, k', v) è presente, lo è anche (k', k, v) .
 - Il numero di righe in `rhs.txt` è esattamente N^2 .
- **Dipendenze:** Modulo 1 (`coords.txt`); Modulo 2 (`ordering.txt`, solo se `-reorder`).
- **Complessità attesa:** $\mathcal{O}(N^2)$ in tempo (al più 5 entrate per nodo) e spazio.

Logica implementativa (pseudo-codice)

1. Leggi N e flag `--reorder`
2. Carica `coords.txt` in mappa `coords[id] = {i, j, x, y}`
3. Se `--reorder`: carica `ordering.txt`, costruisci `perm` e `inv_perm`
4. altrimenti: `perm[k] = inv_perm[k] = k`
5. Scegli condizione al bordo u_0 (menu: scelte 0..5)
6. Per $k = 0 \dots N^2 - 1$:
7. `old_id = perm[k]`; leggi (i, j, x, y) da `coords[old_id]`
8. Aggiungi triplet $(k, k, -4*\kappa/h^2)$
9. `b[k] = -f(x, y)`
10. Per ogni direzione d in $\{su, giu, sx, dx\}$:
11. $(ni, nj) = \text{vicino di } (i, j) \text{ in direzione } d$
12. Se (ni, nj) fuori dominio:
13. `b[k] -= (kappa/h^2) * u0(ni*h, nj*h)`
14. Altrimenti:
15. `k_vic = inv_perm[id_naturale(ni, nj)]`
16. Aggiungi triplet $(k, k_vic, +\kappa/h^2)$
17. Scrivi `A.txt` (triplet) e `rhs.txt` (vettore b)

Condizioni al bordo disponibili (parte opzionale)

Scelta	Funzione $u_0(x, y)$	Utilità di verifica
0	0	Caso base omogeneo (default)
1	10	Con $f = 0$, la soluzione deve essere $\equiv 10$
2	x	Con $f = 0$, la soluzione interna deve coincidere con x
3	$\sin(\pi x)$	Bordo continuo; si annulla agli angoli
4	$x^2 - y^2$	$\nabla^2(x^2 - y^2) = 0$: soluzione numerica esatta
5	$\mathbf{1}_{y=1}$	Shock termico: solo il bordo superiore è caldo

Verifica. Con $N = 4$ si confronta $\Delta b_k = b_k^{(u_0)} - b_k^{(0)}$ calcolato dal programma con il valore teorico atteso per ogni nodo adiacente al bordo. I 16 nodi adiacenti al bordo devono restituire un errore $< 10^{-10}$; i 4 nodi interni devono avere $\Delta b_k = 0$.

Modulo 4 – `solver.ipynb`: risoluzione e analisi numerica

Il Modulo 4 è il punto terminale della pipeline. Legge i file prodotti dal Modulo 3, esegue la fattorizzazione di Cholesky e risolve il sistema lineare. Include anche la fase di analisi sperimentale comparativa tra le due strategie di ordinamento.

Specifiche

- **Input:**
 - `output/A.txt` e `output/rhs.txt` (prodotti dal Modulo 3).
 - Parametro N (o serie $N \in \{32, 64, 128, 256, 512, 1024\}$ per il benchmark).
- **Output:**
 - Vettore soluzione $u \in \mathbb{R}^{N^2}$, rimappato sulla griglia originale per la visualizzazione.
 - Spy plot della matrice A e del fattore L per ordinamento naturale e nested dissection.
 - Mappa di calore 2D e superficie 3D della soluzione $u(x, y)$.
 - Grafico log-log del fill-in `nnz(L)` e dei tempi di fattorizzazione al variare di N .
- **Funzionalità richieste:**
 - Lettura di `A.txt` e costruzione di `scipy.sparse.coo_matrix`, con conversione in `csc_matrix`.
 - Fattorizzazione $-A = LL^T$ tramite `sksparse.cholmod.cholesky` con `order="natural"`.
 - Risoluzione del sistema con `scipy.sparse.linalg.spsolve_triangular` (forward e backward substitution).
 - Automazione dell'intera pipeline C++ tramite `subprocess` per ciascun valore di N e ordinamento.
 - Misurazione dei tempi con `time.perf_counter`.
- **Strutture dati:**

- `scipy.sparse.coo_matrix` per l'assemblaggio iniziale; `csc_matrix` per la fattorizzazione.
- `numpy.ndarray` per la soluzione u e il termine noto b .
- Oggetto `Factor` di `CHOLMOD` per estrarre la matrice L esplicita.
- **Invarianti:**
 - La matrice costruita da `coo_matrix` deve essere simmetrica e definita negativa.
 - Il vettore soluzione u deve soddisfare $\|Au - b\|_2 / \|b\|_2 < 10^{-10}$ (residuo relativo).
- **Dipendenze:** Modulo 3 (`A.txt`, `rhs.txt`); librerie `scipy`, `numpy`, `scikit-sparse`, `matplotlib`.
- **Complessità attesa:** Fill-in $O(N^3)$ con ordinamento naturale; $O(N^2 \log N)$ con nested dissection. Tempo di fattorizzazione dominato dal fill-in.

Logica implementativa (pseudo-codice)

1. Per N in $\{32, 64, 128, 256, 512, 1024\}$:
2. Per `ordering` in $\{\text{naturale}, \text{nested_dissection}\}$:
3. Esegui pipeline C++ via `subprocess`
4. Leggi `A.txt` -> costruisci `coo_matrix` -> converti in `csc_matrix`
5. Leggi `rhs.txt` -> array `numpy b`
6. Avvia timer
7. `factor = cholesky(-A, order="natural")`
8. `L = csc_matrix(factor[0])`
9. `y = spsolve_triangular(L, b, lower=True)` # forward
10. `u = spsolve_triangular(L.T, y, lower=False)` # backward
11. Ferma timer; registra tempo e `nnz(L)`
12. Genera grafici: `spy plot`, `heatmap`, `superficie 3D`, `log-log benchmark`

4 Piano di validazione sperimentale

L'obiettivo della validazione è quantificare il vantaggio computazionale dell'ordinamento *nested dissection* rispetto all'ordinamento naturale, e verificare la correttezza fisica della soluzione numerica.

Metriche di confronto

Metrica	Ordinamento naturale	Nested dissection
Fill-in $\text{nnz}(L)$	$O(N^3)$	$O(N^2 \log N)$
Tempo fattorizzazione	$O(N^4)$	$O(N^3)$
Larghezza di banda di A	$\Theta(N)$	$O(\sqrt{N^2}) = O(N)$

Test di correttezza

- **Residuo relativo:** Per ogni N testato, verificare $\|Au - b\|_2 / \|b\|_2 < 10^{-10}$.
- **Soluzione esatta con $u_0 = x^2 - y^2$:** Poiché $\nabla^2(x^2 - y^2) = 0$, con $f = 0$ la soluzione numerica deve coincidere con u_0 fino agli errori di arrotondamento.
- **Verifica della permutazione:** Il vettore `ordering` deve essere una permutazione valida di $\{0, \dots, N^2 - 1\}$.

Test di scalabilità

Il benchmark viene eseguito per $N \in \{32, 64, 128, 256, 512, 1024\}$. I risultati ottenuti sono visualizzati in scala log-log:

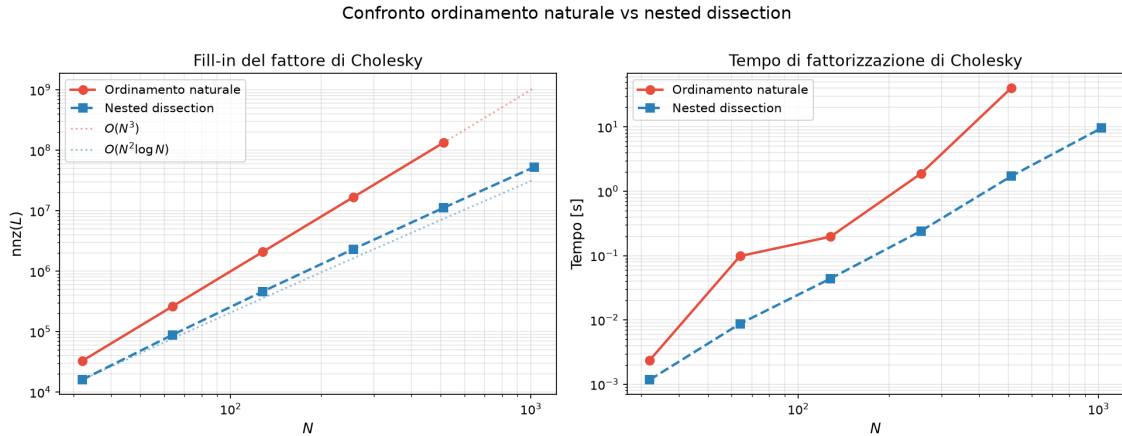


Figura 2: Scala log-log di $\text{nnz}(L)$ e tempo di fattorizzazione al variare di N , con e senza riordinamento.

Come visibile in Figura 2, il numero di entrate non nulle nel fattore L mostra due andamenti divergenti: l'ordinamento naturale segue la pendenza $O(N^3)$, mentre la nested dissection segue la pendenza più piatta $O(N^2 \log N)$, con un risparmio che diventa dominante per $N \geq 128$.

Nota sui limiti hardware ($N = 1024$): Per il caso estremo $N = 1024$ (circa un milione di incognite), l'algoritmo con ordinamento Naturale satura rapidamente la memoria RAM disponibile (richiedendo oltre 6.5 GB fisici per allocare il fattore L), sfociando di fatto in un blocco del sistema operativo per Out-Of-Memory. Pertanto, la curva dell'ordinamento Naturale

si interrompe in Figura 2 a $N = 512$. Questo ostacolo fisico tangibile evidenzia in modo inequivocabile la necessità ingegneristica di ricorrere ad algoritmi ottimali come la *Nested Dissection*, che completa in meno di 10 secondi lo stesso problema da un milione di equazioni allocando una frazione trascurabile di RAM.

Visualizzazioni

1. **Spy plot:** Struttura di sparsità della matrice A e del fattore L per $N = 32$, per entrambi gli ordinamenti. Si deve osservare un blocco quasi-pieno per l'ordinamento naturale e una struttura a blocchi sparsi per la nested dissection.
2. **Mappa di calore e superficie 3D:** La soluzione $u(x, y)$ deve mostrare un picco di temperatura in prossimità dell'angolo $(0, 0)$, dove la sorgente gaussiana è concentrata, con decadimento verso i bordi freddi.